

OOP Project Report 2567

หัวข้อ: Pizza Website

ชื่อกลุ่ม: Durain

รายชื่อสมาชิก:

1. พงศภักดิ์ ต๊ะตังใจ 66011428
2. พัฒน์กุลธร ชัยรัตน์ 66011437
3. ราธา โรจน์รุจิพงศ์ 66011464

บทนำ:

หัวข้อของกลุ่มเราคือการทำเว็บไซต์สั่งพิซซ่าออนไลน์ โดยมีต้นแบบจาก <https://1112.com/> เว็บไซต์ของเราจะเริ่มจากหน้า login ก่อน ถ้ากรอก username และ password ถูกต้องก็จะดึงมาหน้า menu ที่แสดงรายการอาหารชนิดต่างๆอยู่ แต่ถ้าผู้ใช้ยังไม่มีบัญชีก็สามารถ sign up ได้ หลังจากผ่านหน้า login แล้ว ผู้ใช้สามารถมาดูอาหารที่สั่งได้และกดเพิ่มลงตะกร้าตามใจชอบ และเมื่อซื้อจนหน้าจ๋าแล้ว ก็สามารถกดยืนยันของในตะกร้าได้ โดยต้องกรอกข้อมูลจัดส่ง เช่น ที่อยู่จัดส่ง หรือรับเองที่ร้าน พร้อมสาขาที่ต้องการ รวมถึงการใช้คูปองลดราคาอาหาร จากนั้นก็จะดึงไปหน้า confirm payment เพื่อจ่ายเงิน นอกเหนือจากนี้ผู้ใช้แต่ละคนเอง สามารถมาเขียนรีวิวให้กับพิซซ่าได้ด้วย เพื่อรับคูปองส่วนลดเพิ่มเติม ในส่วนของร้านค้าเอง ก็จะมี username และ password สำหรับ role Admin ร้านละหนึ่ง ID โดยทำได้อยู่ 2 อย่าง 1. เช็คสต็อกในแต่ละสาขาว่าเหลือเท่าไรบ้าง 2. รีสต็อก เพื่อเพิ่มของในสาขาที่ประจำอยู่ กรณีของใกล้เคียง

หน้าจอของ

1

MyPizza

Home Basket Menu Transaction Review

Login Sign up

Please sign up

Username

Password

Confirm Password

Sign in

Don't have an account yet? [Click here](#) to sign up

2

MyPizza

Home Basket Menu Transaction Review

Login Sign up



Please sign in

Username

Password

Sign in

Don't have an account yet? [Click here](#) to sign up

3

MyPizza

[Home](#) [Basket](#) **Menu** [Transaction](#) [Review](#)

[Login](#)

[Sign-up](#)



PIZZA



DRINKS



4

MyPizza

[Home](#) [Basket](#) [Menu](#) [Transaction](#) [Review](#)

[Login](#)

[Sign-up](#)

Your Custom



Choose

Size:

M



Quantity:

1

[Add To Basket](#)

5

MyPizza

[Home](#) [Basket](#) [Menu](#) [Transaction](#) [Review](#)

[Login](#)

[Sign-up](#)

Your Basket

Stock List

Item	Size	Quantity
hawaiian	M	1
margherita	L	1

Drink List

Item	Size	Quantity
coke	L	2

More? Add more!!!

OK? goto Order!!! Let's Go!!!

6

MyPizza

[Home](#) [Basket](#) [Menu](#) [Transaction](#) [Review](#)

[Login](#)

[Sign-up](#)

Your Order

Stock List

Item	Size	Quantity
hawaiian	M	1
margherita	L	1

Drink List

Item	Size	Quantity
coke	L	2

Total Price: 414

Apply Coupon

-10 Bath ▼

Select Delivery

- ☐ Delivery
☐ Pick-Up

Select Shop

TUR Shop ▼

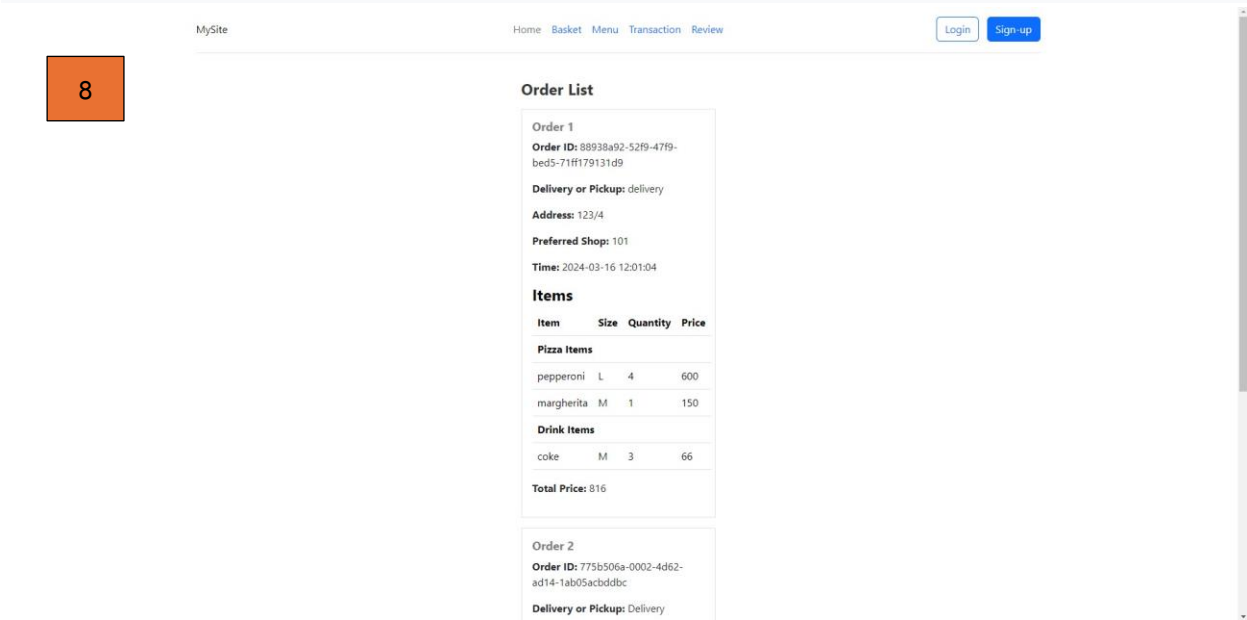
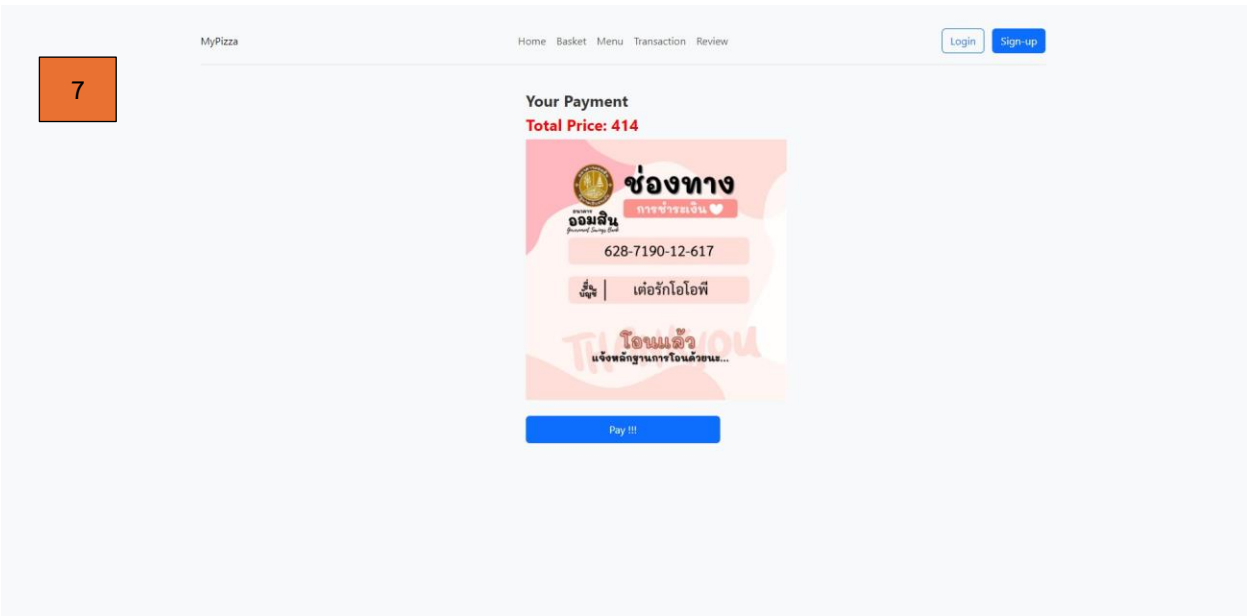
Enter Your Address

Enter your address

Select Payment Method

- ☐ Credit Card
☐ QR Code

Confirm Order



9

MyPizza

Home Basket Menu Transaction Review

LoginSign up

We need your review
Thank you!!!

Review

Rating:

0★

Submit Review

Review Token: 2

User ID: tur
Rating: ★★★★★
Comment: good
Time: 2024-03-16 12:01:04

User ID: tur
Rating: ★★★★★☆
Comment: not bad
Time: 2024-03-16 12:01:04

User ID: tur

0★

Submit Review

Review Token: 1

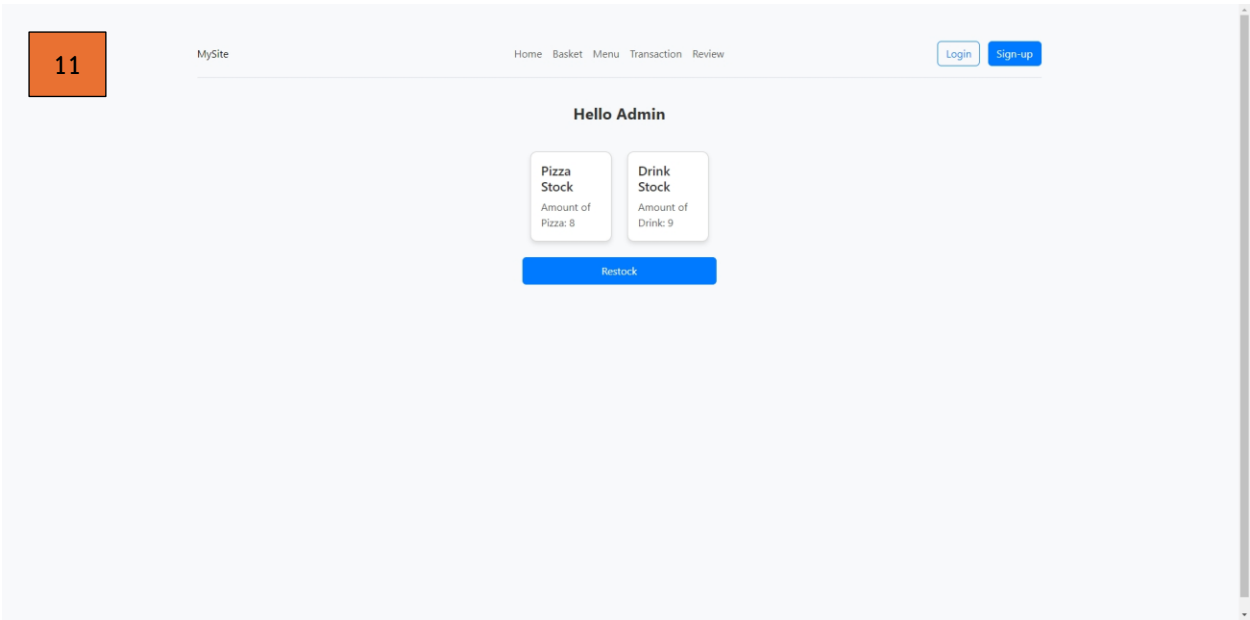
User Name: tur
Rating: ★★★★★
Comment: good
Time: 2024-03-16 12:21:17

User Name: tur
Rating: ★★★★★☆
Comment: not bad
Time: 2024-03-16 12:21:17

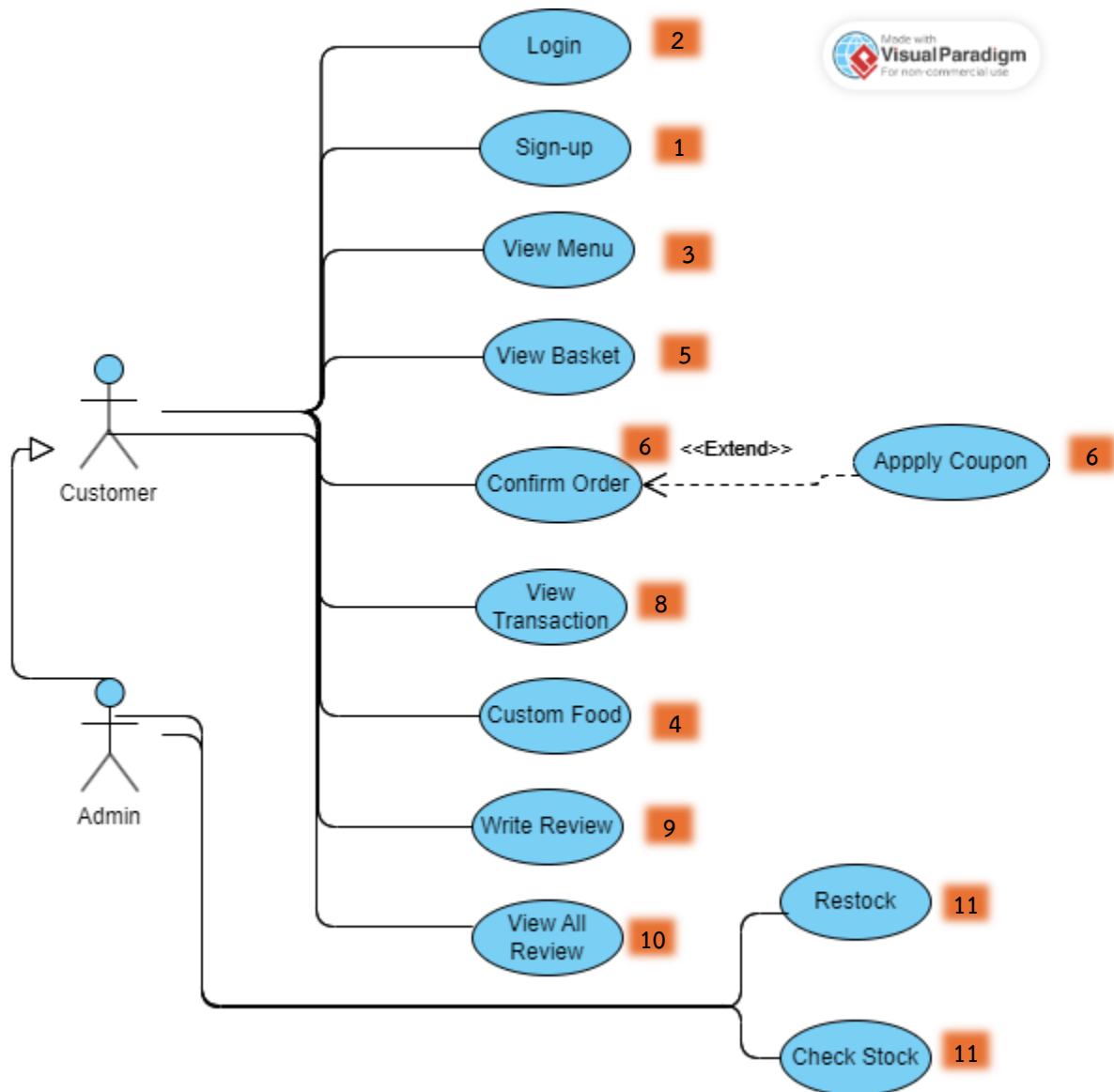
User Name: tur
Rating: ★★★★★☆
Comment: mai aroi loey
Time: 2024-03-16 12:21:17

User Name: tart
Rating: ★★★★★☆
Comment: delicious pizza
Time: 2024-03-16 12:21:17

gotoHomepage



Use Case Diagram:



Use Case Name	Login
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานกดปุ่มเข้าสู่ระบบ
Pre-Condition	เข้าหน้าเว็บไซต์
Post-Condition	เข้าสู่ระบบสำเร็จ

Use Case Name	Sign-up
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานกรอก username และ password
Pre-Condition	เข้าหน้าเว็บไซต์
Post-Condition	มี user id เป็นของตัวเอง

Use Case Name	View Menu
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานกดเลือกดูและเลือกอาหารที่ชอบ
Pre-Condition	เข้าสู่ระบบสำเร็จแล้ว
Post-Condition	เลือกรายละเอียดอาหาร (custom)

Use Case Name	Custom Food
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานกดเลือกรายละเอียดอาหาร (ขนาด, จำนวน)
Pre-Condition	กดเลือกรูปอาหารที่ต้องการ
Post-Condition	อาหารลงตะกร้า และกลับหน้าเมนู

Use Case Name	View Basket
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานกดดูตะกร้าสินค้าตัวเอง
Pre-Condition	เข้าสู่ระบบสำเร็จแล้ว
Post-Condition	แสดงหน้าตะกร้า

Use Case Name	Confirm Order
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานกด confirm order สินค้าในตะกร้า
Pre-Condition	เลือกอาหารที่ต้องการลงตะกร้า
Post-Condition	แสดงหน้า confirm order

Use Case Name	Apply Coupon
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานกดคูปองที่ต้องการจะใช้ลดราคาอาหาร
Pre-Condition	ต้องมีคูปองอยู่แล้ว ถึงจะกดได้
Post-Condition	ลดราคาอาหารสำเร็จ

Use Case Name	View Transaction
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานกดดู Transaction ที่เคยสั่งอาหารไปทั้งหมด
Pre-Condition	เข้าสู่ระบบสำเร็จ
Post-Condition	แสดงหน้า Transaction

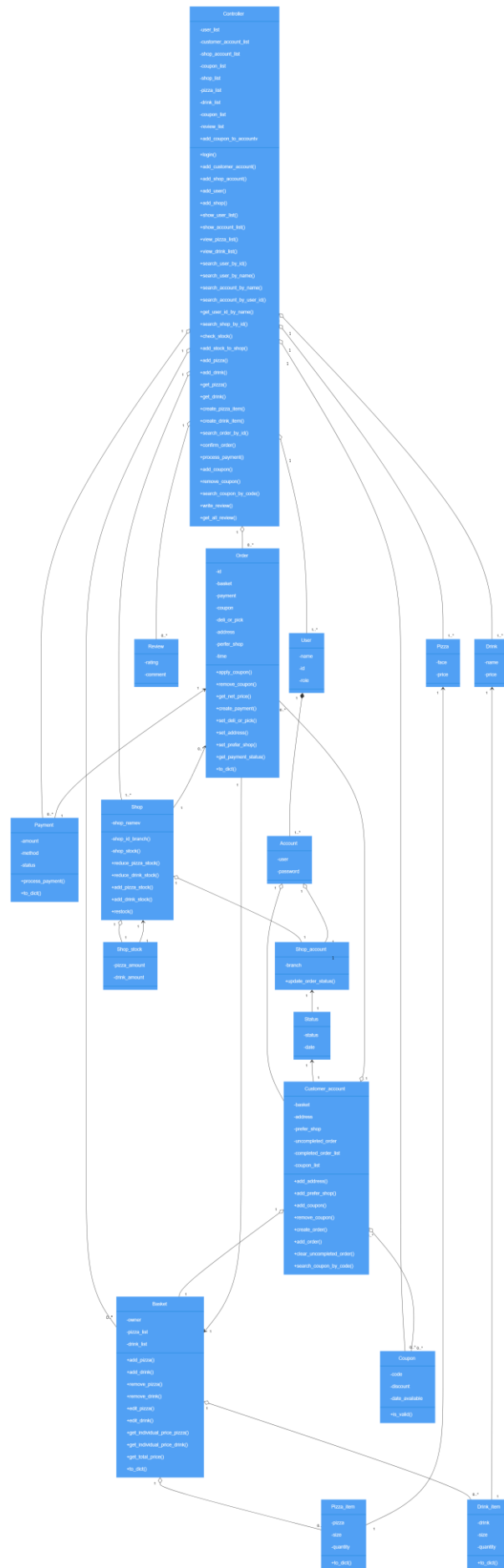
Use Case Name	Write Review
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานเขียนคอมเมนต์ให้พิกซ์ซ่า
Pre-Condition	เข้าสู่ระบบสำเร็จ และเคยสั่งอาหารมาก่อน (1 order ต่อ 1 รีวิว)
Post-Condition	รีวิวสำเร็จ

Use Case Name	View All Review
Actors	ผู้ใช้ทั่วไปและผู้ดูแลระบบ
Descriptions	ผู้ใช้งานกดดูรีวิวทั้งหมดที่เคยมีมา ของทุกผู้ใช้งาน
Pre-Condition	เข้าสู่ระบบสำเร็จ
Post-Condition	แสดงหน้ารีวิว

Use Case Name	Restock
Actors	ผู้ดูแลระบบ
Descriptions	ผู้ดูแลระบบกดเติมสินค้าเข้าร้านค้าที่ประจำอยู่
Pre-Condition	เข้าสู่ระบบสำเร็จ
Post-Condition	เติมสินค้าลงร้านค้าสำเร็จ

Use Case Name	Check Stock
Actors	ผู้ดูแลระบบ
Descriptions	ผู้ดูแลระบบกดดูจำนวนสินค้าที่คงเหลืออยู่ ณ ร้านค้าที่ประจำอยู่
Pre-Condition	เข้าสู่ระบบสำเร็จ
Post-Condition	แสดงจำนวนสินค้า

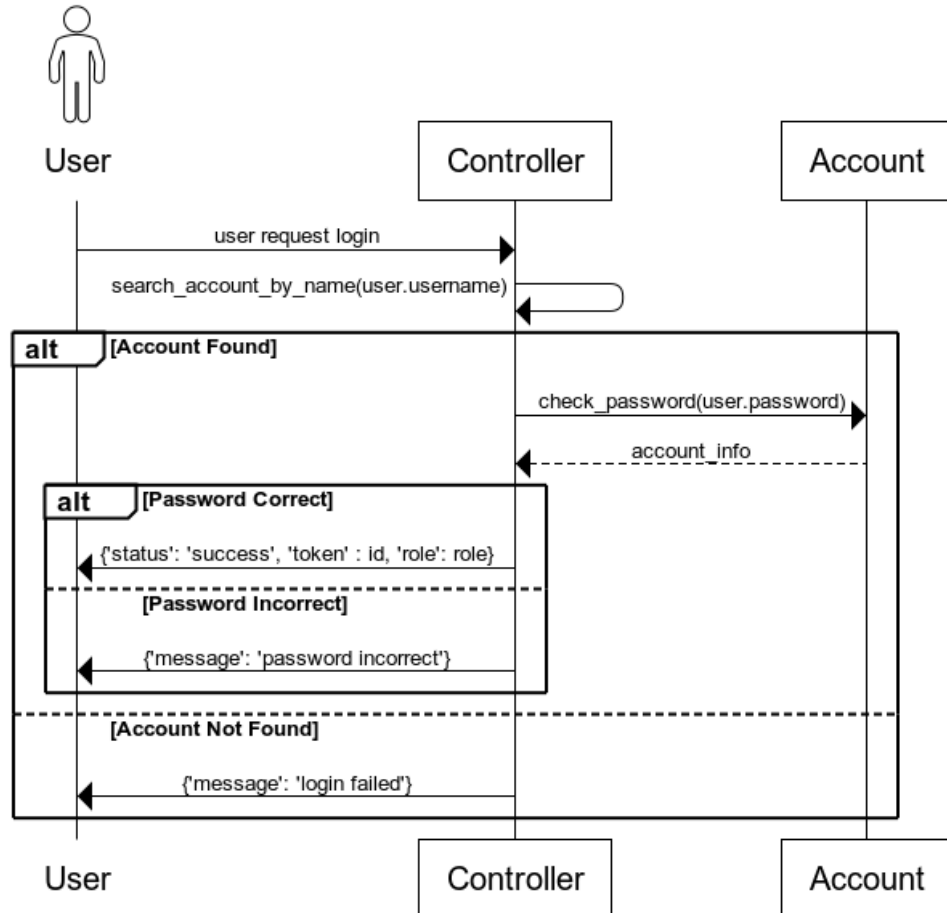
Class Diagram:



Sequence Diagram:

1. เข้าสู่ระบบ (Login)

User Login Sequence



www.websequencediagrams.com

```

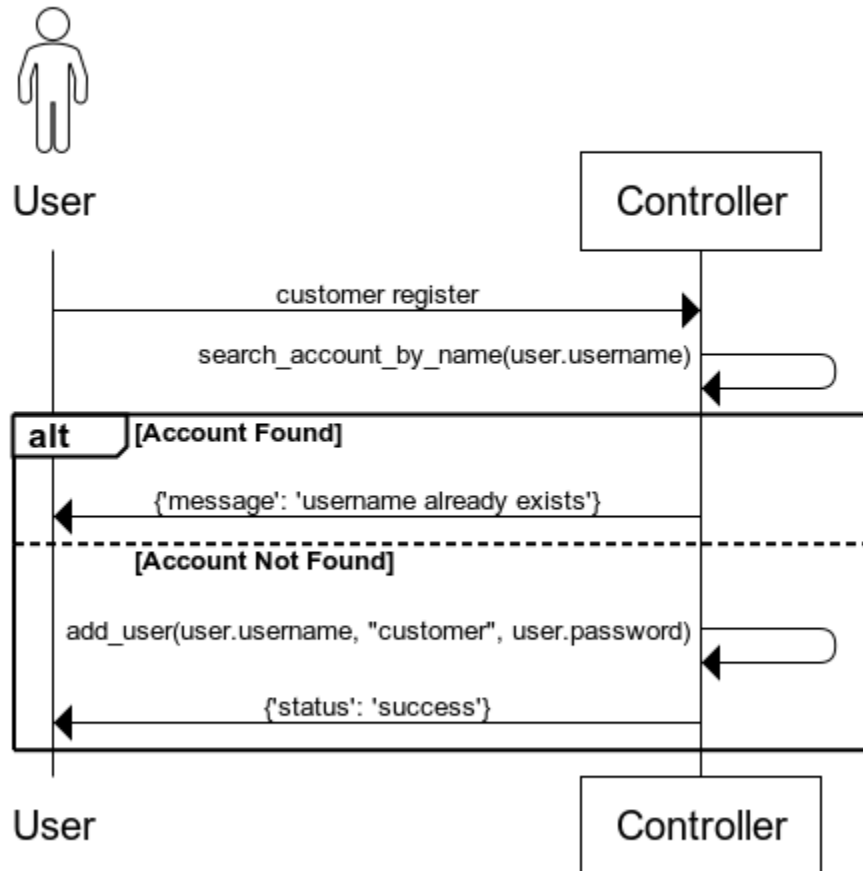
@app.post('/login/')
async def login(user: loginDTO) -> dict:
    account = controller.search_account_by_name(user.username)
    if account:
        if account.password == user.password:
            id = account.user.id
            role = account.user.role
            return {'status': 'success', 'token': id, 'role': role}
        else:
            return {"message": "password incorrect"}
    else:
        return {"message": "login failed" }
  
```

```

def search_account_by_name(self, name: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.name == name:
            return account
    return None
  
```

2. ลงทะเบียน (Register)

User Registration Sequence



www.websequencediagrams.com

```

@app.post('/register/', tags=["user"])
async def register(user: loginDTO) -> dict:
    if controller.search_account_by_name(user.username):
        return {"message": "user name already exist"}
    controller.add_user(user.username, "customer", user.password)
    return {'status': 'success'}
  
```

```

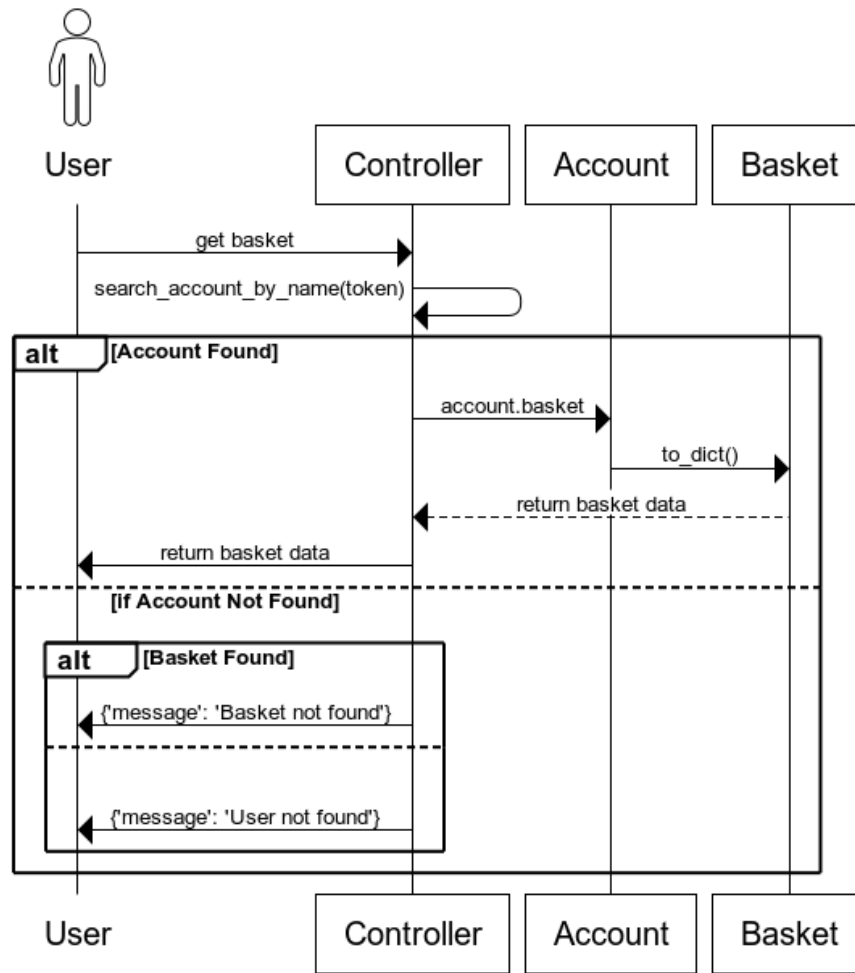
def search_account_by_name(self, name: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.name == name:
            return account
    return None
  
```

```

def add_user(self, name: str, role: str, password: str) -> None:
    self.__user_list.append(User(name, role))
    if role == "customer":
        self.add_customer_account(name, password)
    elif role == "shop":
        self.add_shop_account(name, password)
  
```

3. ดูของในตะกร้า (Get basket)

Get Basket Sequence



www.websequencediagrams.com

```

@app.get('/basket/', tags=["basket"])
def get_basket(token: str) -> dict:
    account = controller.search_account_by_name(token)
    # account = controller.search_account_by_user_id(token)
    if account != None :
        if account.basket :
            return account.basket.to_dict()
        else:
            return {"message" : "Basket not found"}
    return {"message" : "User not found"}
  
```

```

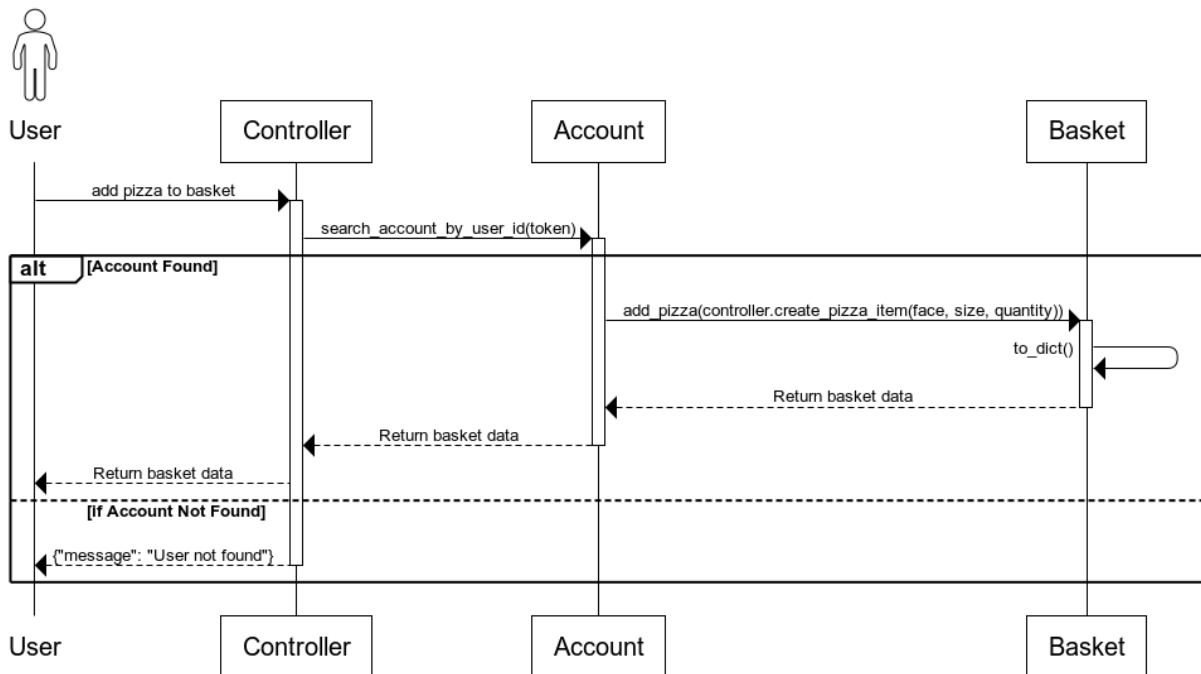
def search_account_by_name(self, name: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.name == name:
            return account
    return None
  
```

```

def to_dict(self) -> dict:
    pizza_list = [{"item": pizza_item.to_dict(), "price": self.get_individual_price_pizza(pizza_item)} for pizza_item in self.__pizza_list]
    drink_list = [{"item": drink_item.to_dict(), "price": self.get_individual_price_drink(drink_item)} for drink_item in self.__drink_list]
    return {"pizza_list": pizza_list, "drink_list": drink_list}
  
```

4. เพิ่มพิซซ่าลงตะกร้า (Add pizza to basket)

Add Pizza to Basket Sequence



www.websequencediagrams.com

```

@app.post('/basket/add_pizza/', tags=["basket"])
def add_pizza_to_basket(token: str, face: str, size: str, quantity: int) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    account.basket.add_pizza(controller.create_pizza_item(face, size, quantity))
    return account.basket.to_dict()
  
```

```

def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

```

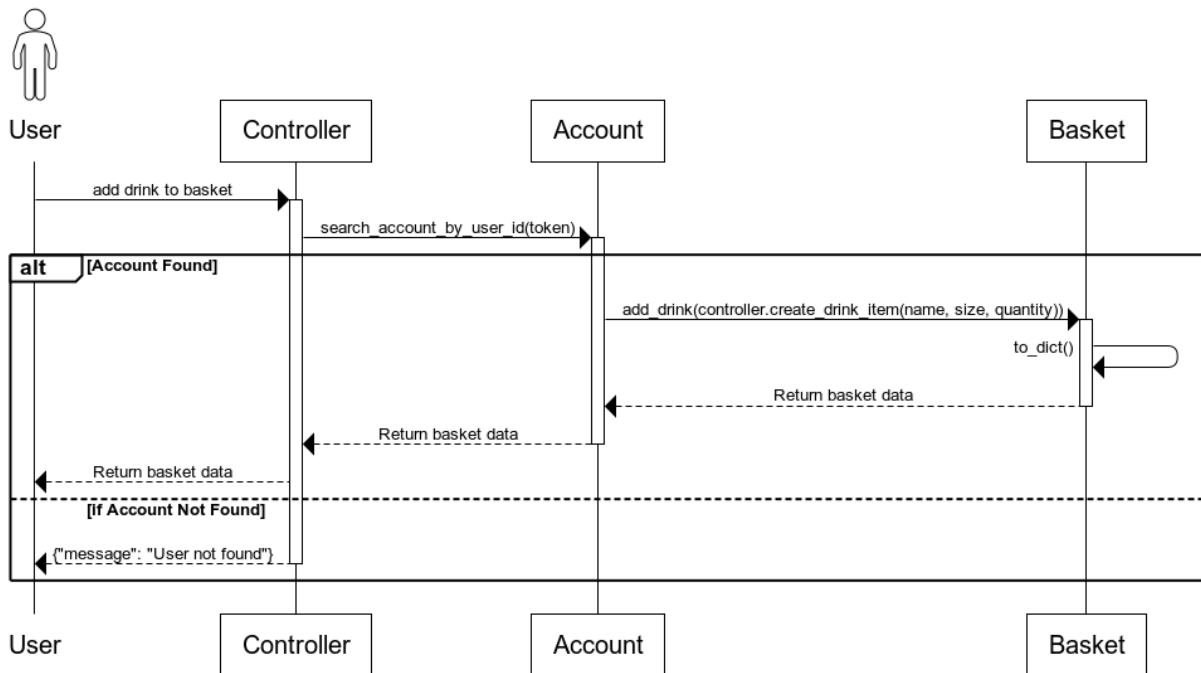
def add_pizza(self, item: Pizza_item) -> None:
    for pizza in self.__pizza_list:
        if item.pizza == pizza.pizza and item.size == pizza.size:
            pizza.quantity += item.quantity
            break
    else:
        self.__pizza_list.append(item)
  
```

```

def to_dict(self) -> dict:
    pizza_list = [{"item": pizza_item.to_dict(), "price": self.get_individual_price_pizza(pizza_item)} for pizza_item in self.__pizza_list]
    drink_list = [{"item": drink_item.to_dict(), "price": self.get_individual_price_drink(drink_item)} for drink_item in self.__drink_list]
    return {"pizza_list": pizza_list, "drink_list": drink_list}
  
```


5. เพิ่มเครื่องดื่มลงตะกร้า (Add drink to basket)

Add Drink to Basket Sequence



www.websequencediagrams.com

```

@app.post('/basket/add_drink/', tags=["basket"])
def add_drink_to_basket(token: str, name: str, size: str, quantity: int) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    account.basket.add_drink(controller.create_drink_item(name, size, quantity))
    return account.basket.to_dict()
  
```

```

def add_drink(self, item: Drink_item) -> None:
    for drink in self.__drink_list:
        if item.drink == drink.drink and item.size == drink.size:
            drink.quantity += item.quantity
            break
    else:
        self.__drink_list.append(item)
  
```

```

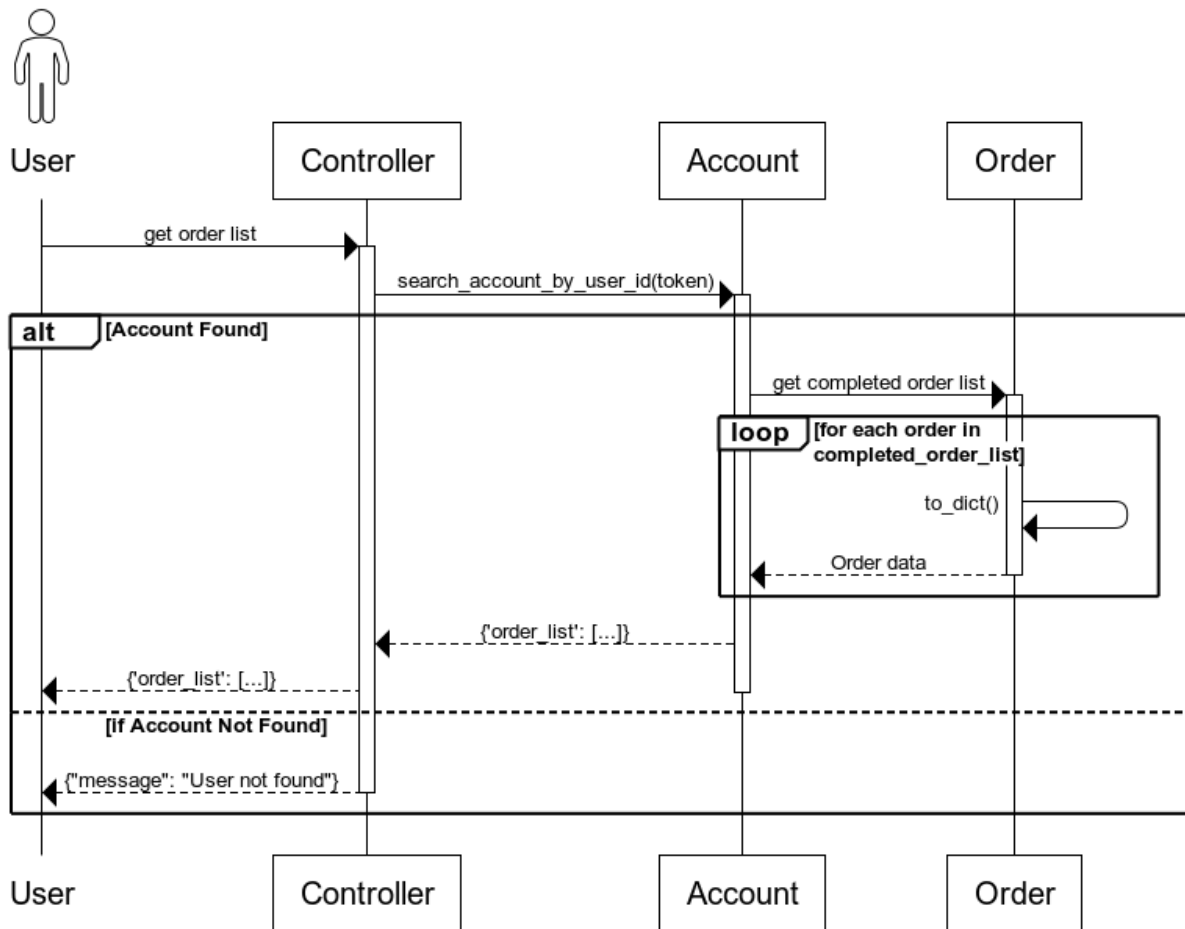
def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

```

def to_dict(self) -> dict:
    pizza_list = [{"item": pizza_item.to_dict(), "price": self.get_individual_price_pizza(pizza_item)} for pizza_item in self.__pizza_list]
    drink_list = [{"item": drink_item.to_dict(), "price": self.get_individual_price_drink(drink_item)} for drink_item in self.__drink_list]
    return {"pizza_list": pizza_list, "drink_list": drink_list}
  
```

6. ดูอาหารที่เคยสั่งทั้งหมด (Get order list)

Get Order List Sequence



www.websequencediagrams.com

```

@app.get('/order/', tags=["order"])
def get_order_list(token: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    return {'order_list':
        [order.to_dict() for order in account.completed_order_list]}
  
```

```

@app.get('/order/', tags=["order"])
def get_order_list(token: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    return {'order_list':
        [order.to_dict() for order in account.completed_order_list]}
  
```

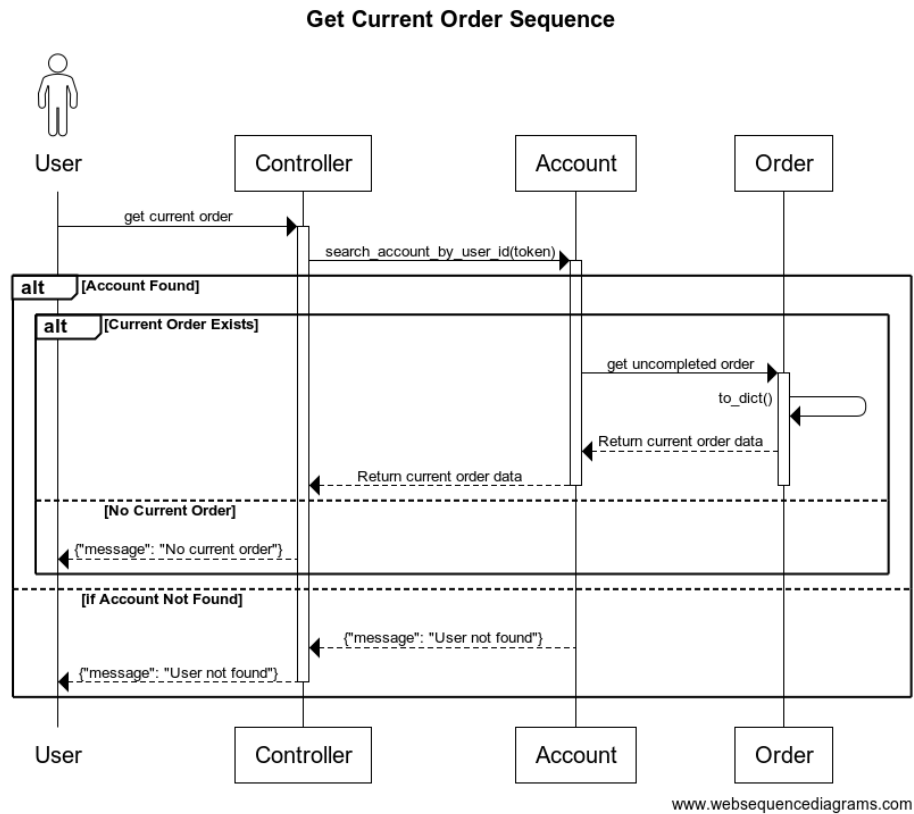
```

def to_dict(self):
    return {
        "order_id": self.__id,
        "basket": self.__basket.to_dict(),
        "coupon": self.__coupon.code if self.__coupon else None,
        "price": self.get_net_price(),
        "deli_or_pick": self.__deli_or_pick,
        "address": self.__address,
        "perfer_shop": self.__perfer_shop,
        "time": self.__time
    }
  
```

```

def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

7. ดูออเดอร์อาหารของปัจจุบัน (Get current order)



```

@app.get('/current_order/', tags=["order"])
def get_current_order(token: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    if account.uncompleted_order == None:
        return {"message" : "No current order"}
    return account.uncompleted_order.to_dict()

```

```

def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None

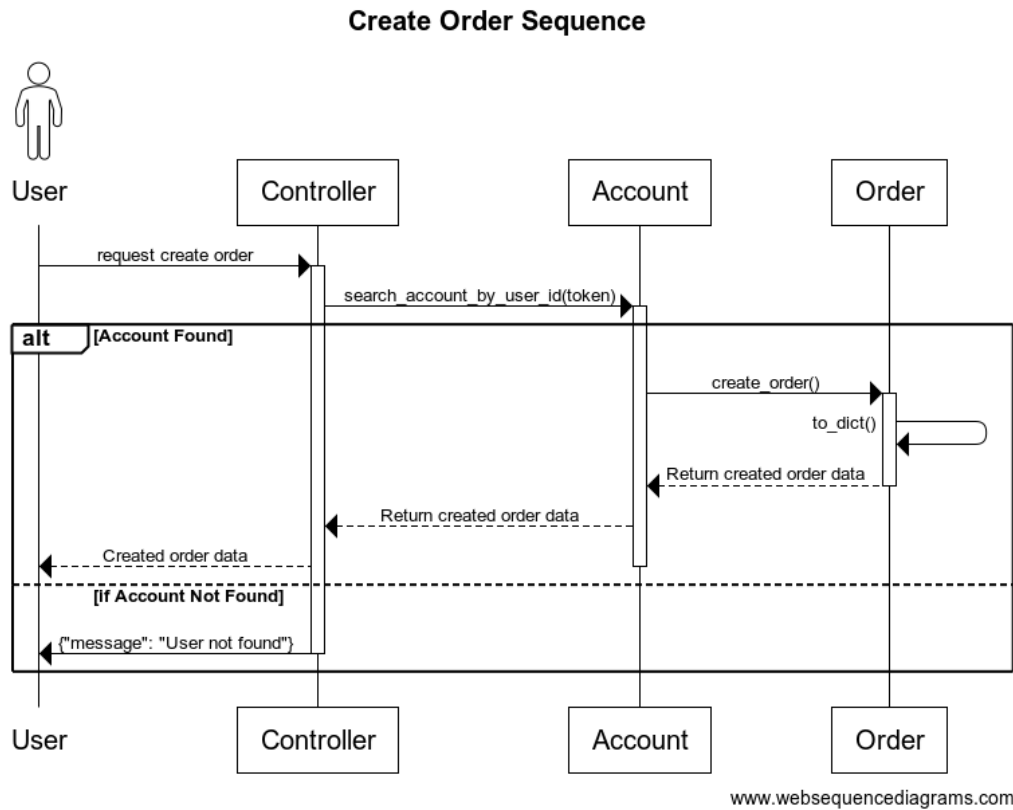
```

```

def to_dict(self):
    return {
        "order_id": self.__id,
        "basket": self.__basket.to_dict(),
        "coupon": self.__coupon.code if self.__coupon else None,
        "price": self.get_net_price(),
        "deli_or_pick": self.__deli_or_pick,
        "address": self.__address,
        "perfer_shop": self.__perfer_shop,
        "time": self.__time
    }

```

8. สร้างออเดอร์อาหาร (Create order)



```

@app.get('/current_order/', tags=["order"])
def get_current_order(token: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    if account.uncompleted_order == None:
        return {"message" : "No current order"}
    return account.uncompleted_order.to_dict()
  
```

```

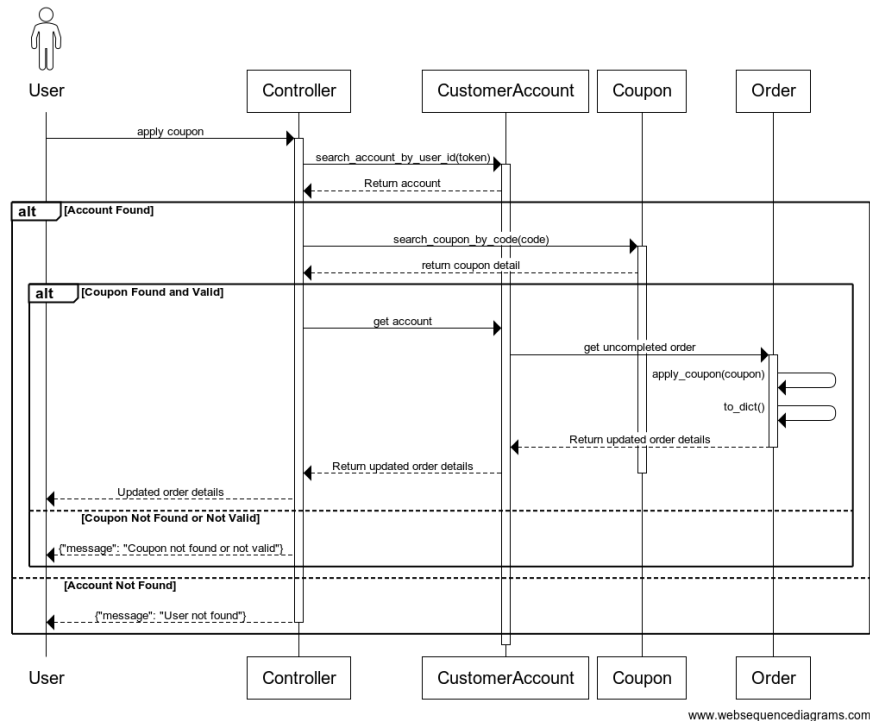
def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

```

def to_dict(self):
    return {
        "order_id": self.__id,
        "basket": self.__basket.to_dict(),
        "coupon": self.__coupon.code if self.__coupon else None,
        "price": self.get_net_price(),
        "deli_or_pick": self.__deli_or_pick,
        "address": self.__address,
        "perfer_shop": self.__perfer_shop,
        "time": self.__time
    }
  
```

9. ใช้งานคูปอง (Apply coupon)

Apply Coupon to Order Sequence



www.websequencediagrams.com

```

@app.post('/order/apply_coupon/', tags=["order"])
def apply_coupon(token: str, code: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    coupon = controller.search_coupon_by_code(code)
    if coupon == None or coupon.is_valid() == False:
        return {"message" : "Coupon not found or not valid"}
    account.uncompleted_order.apply_coupon(coupon)
    return account.uncompleted_order.to_dict()
  
```

```

def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

```

def search_coupon_by_code(self, code: str) -> Coupon:
    for coupon in self.__coupon_list:
        if coupon.code == code:
            return coupon
    raise None
  
```

```

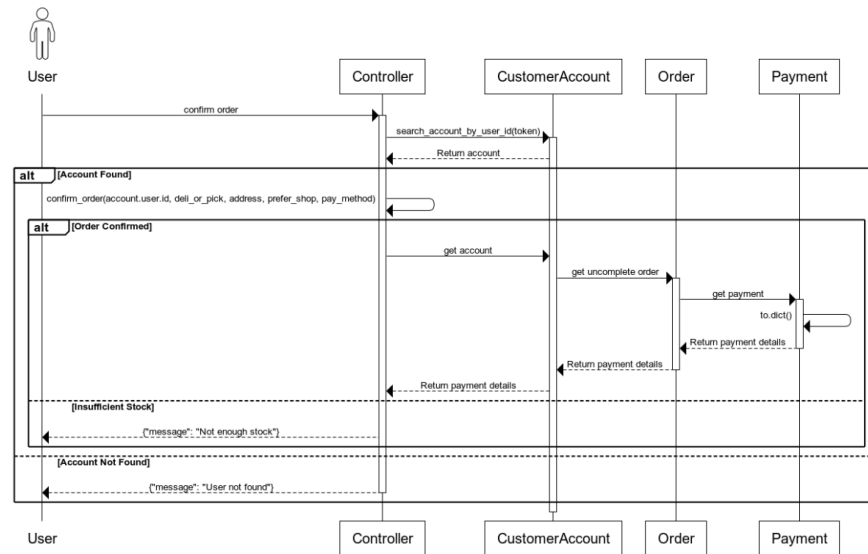
def is_valid(self) -> bool:
    return self.__date_available > time.strftime("%Y-%m-%d")
  
```

```

def to_dict(self):
    return {
        "order_id": self.__id,
        "basket": self.__basket.to_dict(),
        "coupon": self.__coupon.code if self.__coupon else None,
        "price": self.get_net_price(),
        "deli_or_pick": self.__deli_or_pick,
        "address": self.__address,
        "perfer_shop": self.__perfer_shop,
        "time": self.__time
    }
  
```

10. ยืนยันออเดอร์อาหาร (Confirm order)

Confirm Order Sequence



```

@app.post("/order/confirm_order/", tags=["order"])
def confirm_order(token: str, deli_or_pick: str, address: str, prefer_shop: int, pay_method: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}

    if controller.confirm_order(account.user.id, deli_or_pick, address, prefer_shop, pay_method) :
        return account.uncompleted_order.payment.to_dict()
    else :
        return {"message" : "Not enough stock"}
  
```

```

def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

```

def confirm_order(self, user_id: str, deli_or_pick: str, address: str, prefer_shop: int, pay_method: str) -> None:
    account = self.search_account_by_user_id(user_id)
    account.add_address(address)
    account.add_prefer_shop(prefer_shop)
    account.uncompleted_order.set_address(address)
    account.uncompleted_order.set_prefer_shop(prefer_shop)
    account.uncompleted_order.set_deli_or_pick(deli_or_pick)

    pizza_amount = len(account.uncompleted_order.basket.pizza_list)
    drink_amount = len(account.uncompleted_order.basket.drink_list)

    shop = self.search_shop_by_id(prefer_shop)
    pizza_in_stock = shop.shop_stock.pizza_amount
    drink_in_stock = shop.shop_stock.drink_amount

    if pizza_amount > pizza_in_stock or drink_amount > drink_in_stock:
        return False

    shop.shop_stock.set_pizza_amount(pizza_in_stock - pizza_amount)
    shop.shop_stock.set_drink_amount(drink_in_stock - drink_amount)

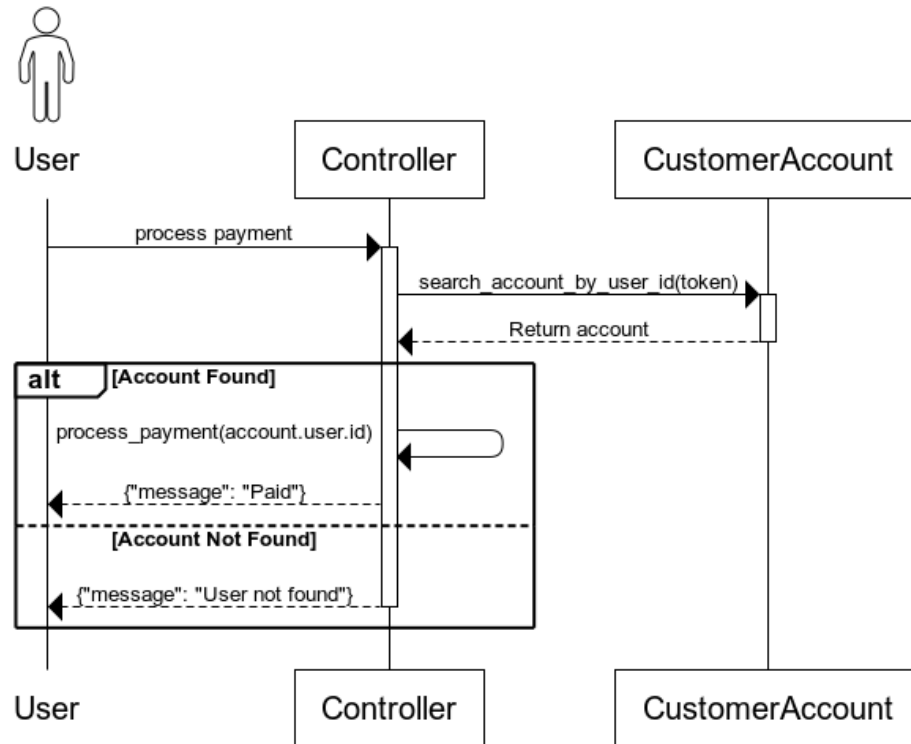
    account.uncompleted_order.create_payment(pay_method)
    return True
  
```

```

def to_dict(self):
    return {
        "amount": self.__amount,
        "method": self.__method,
        "status": self.__status
    }
  
```

11. คำนวณการจ่ายเงิน (Process payment)

Process Payment Sequence



www.websequencediagrams.com

```

@app.post('/order/process_payment/', tags=["payment"])
def process_payment(token: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    controller.process_payment(account.user.id)
    return {"message" : "Paid"}
  
```

```

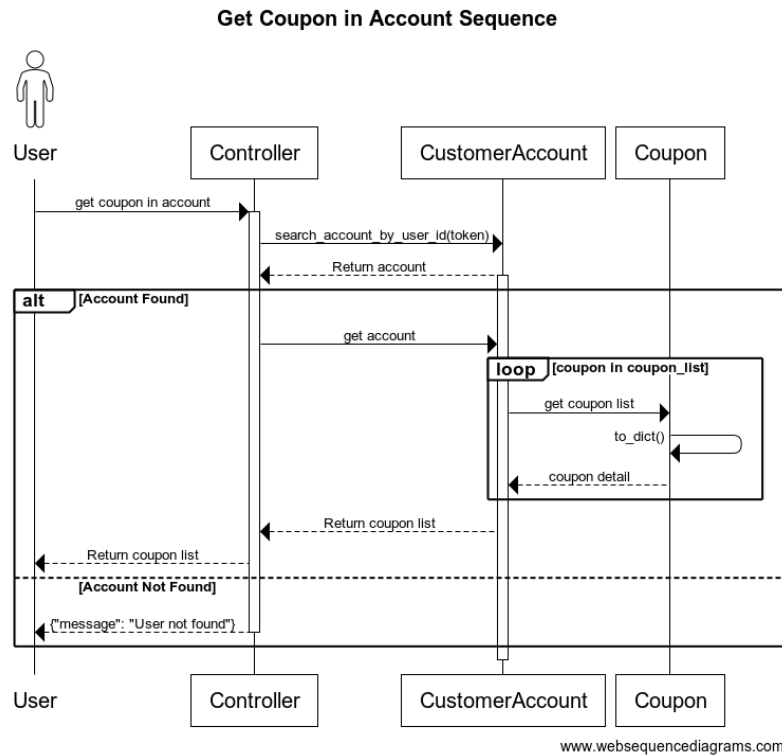
def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

```

def process_payment(self, user_id: str):
    account = self.search_account_by_user_id(user_id)
    account.add_order(account.uncompleted_order)
    account.add_review_token(1)
    account.uncompleted_order.payment.process_payment()

    account.clear_basket()
    account.clear_uncompleted_order()
  
```

12. ตัวอย่างของผู้ใช้งานนั้นๆ (Get coupon in account)



```

@app.get('/coupon/', tags=["coupon"])
def get_coupon_in_account(token: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    return {'coupon_list' : [coupon.to_dict() for coupon in account.coupon_list]}
  
```

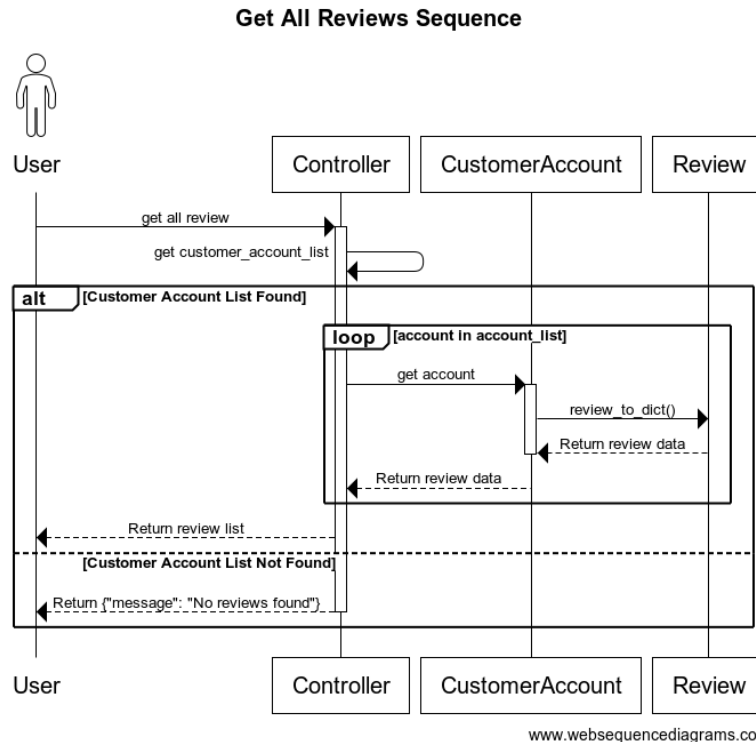
```

def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

```

def to_dict(self) -> dict:
    return {
        "code": self.__code,
        "discount": self.__discount,
    }
  
```


13. ดูรีวิวทั้งหมด (Get all review)



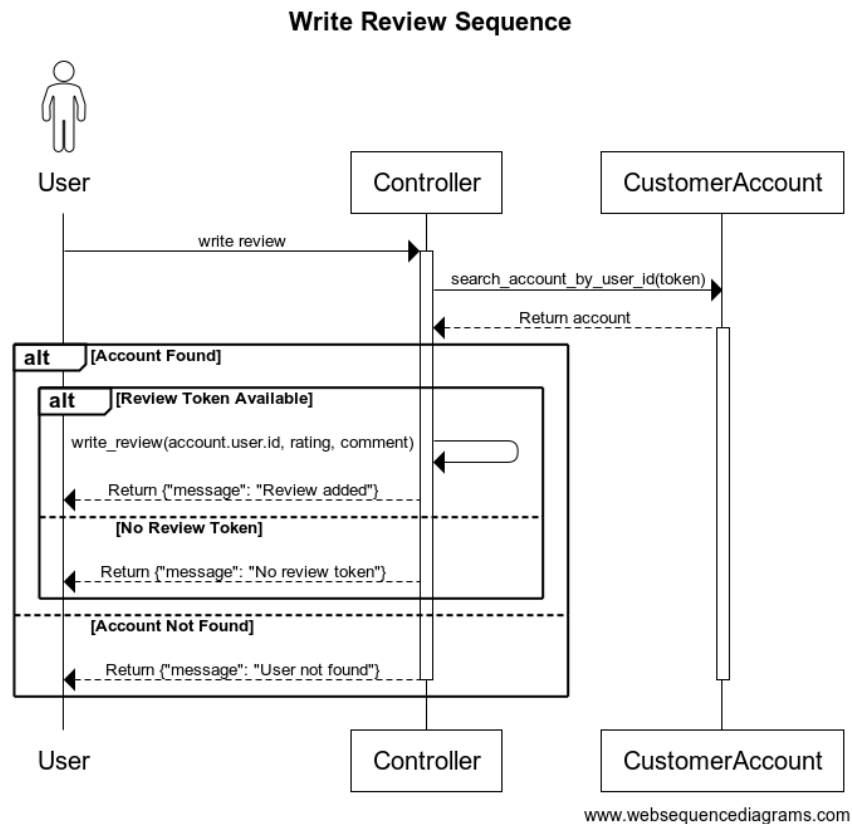
```

@app.get('/review/', tags=["review"])
def get_all_review() -> dict:
    return {'review_list': [account.review_to_dict() for account in controller.customer_account_list]}
  
```

```

def review_to_dict(self) -> list:
    return [{
        'user_id': self.user.name,
        'rating': review.rating,
        'comment': review.comment,
        'time': review.time
    } for review in self._review_list]
  
```

14. เขียนรีวิว (Write review)



```

@app.post('/review/', tags=["review"])
def write_review(token: str, rating: int, comment: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None:
        return {"message": "User not found"}
    if account.review_token == 0:
        return {"message": "No review token"}
    controller.write_review(account.user.id, rating, comment)
    return {"message": "Review added"}
  
```

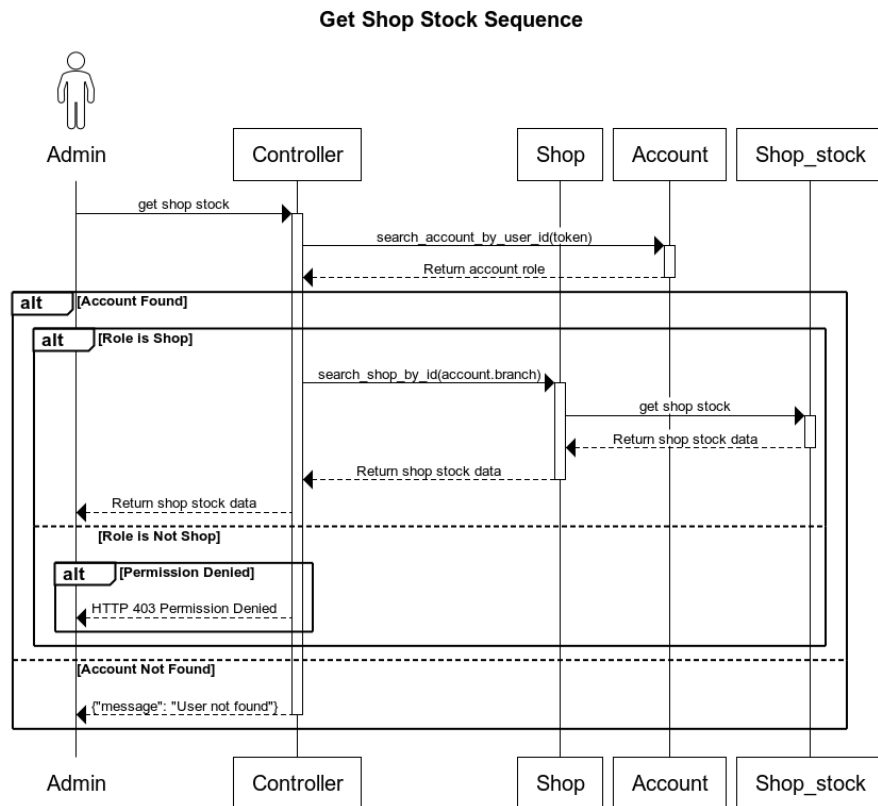
```

def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

```

def write_review(self, user_id: str, rating: int, comment: str) -> None:
    account = self.search_account_by_user_id(user_id)
    if account and account.review_token > 0:
        account.add_review_token(-1)
        review = Review(rating, comment)
        account.add_review(review)
    else:
        return None
  
```

15. ดูคลังสินค้า (Get shops stock [Only Admin])



www.websequencediagrams.com

```

@app.get('/stock/', tags=["stock"])
def get_shops(token: str) -> dict :
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    if account.user.role != "shop":
        raise HTTPException(status_code=403, detail="Permission denied")
    shop = controller.search_shop_by_id(account.branch)
    return shop.shop_stock.to_dict()
  
```

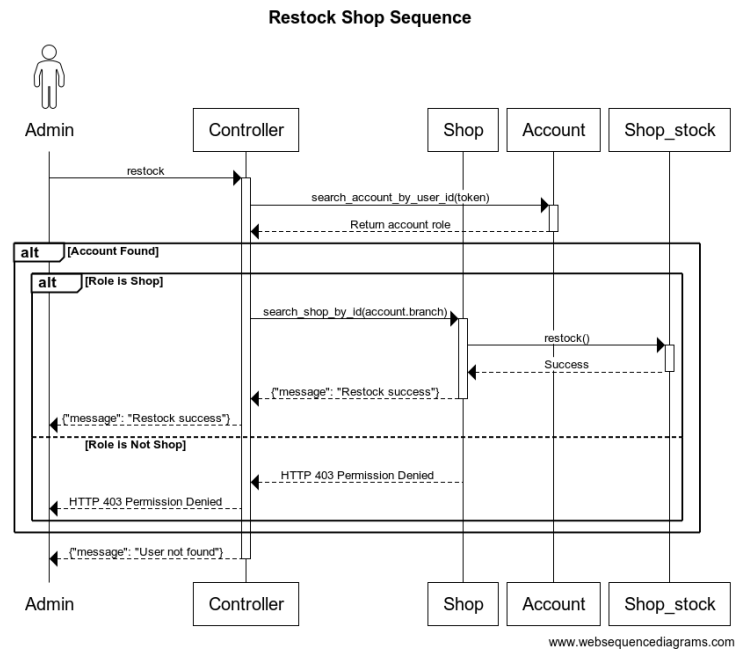
```

def search_account_by_user_id(self, user_id: str) -> Account:
    for account in self.__customer_account_list + self.__shop_account_list:
        if account.user.id == user_id:
            return account
    return None
  
```

```

def search_shop_by_id(self, shop_id_branch: int) -> Shop:
    for shop in self.__shop_list:
        if shop.shop_id_branch == shop_id_branch:
            return shop
    return None
  
```

16. เติมคลังสินค้า (Restock [Only Admin])



```

@app.post('/stock/restock/', tags=["stock"])
def restock(token: str) -> dict:
    # account = controller.search_account_by_name(token)
    account = controller.search_account_by_user_id(token)
    if account == None :
        return {"message" : "User not found"}
    if account.user.role != "shop":
        raise HTTPException(status_code=403, detail="Permission denied")
    shop = controller.search_shop_by_id(account.branch)
    shop.restock()
    return {"message" : "Restock success"}
  
```

```

def search_shop_by_id(self, shop_id_branch: int) -> Shop:
    for shop in self.__shop_list:
        if shop.shop_id_branch == shop_id_branch:
            return shop
    return None
  
```

```

def restock(self) -> None:
    self.__shop_stock.set_pizza_amount(Shop_stock.dially_stock)
    self.__shop_stock.set_drink_amount(Shop_stock.dially_stock)
  
```